

《计算机系统基础》

第2章 信息的表示和处理

任课教师:

何璐璐

luluhe@whu.edu.cn

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- 整数
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

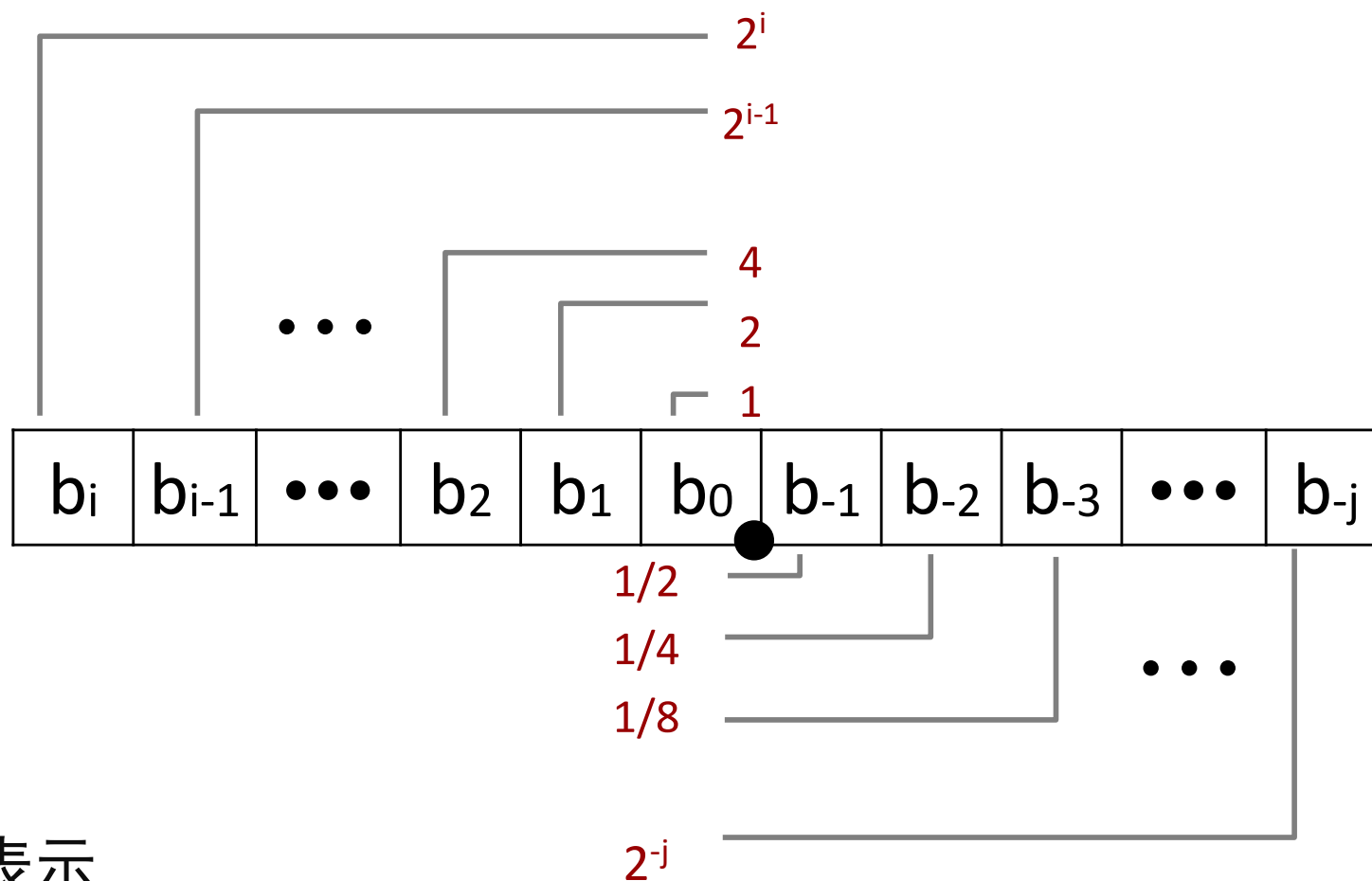
本节内容：浮点数

- 背景：二进制小数
- IEEE 浮点标准：定义
- 示例和性质
- 舍入、加法、乘法
- C语言中的浮点数
- 小结

二进制小数

■ 1011.101_2 是什么？

二进制小数



■ 表示

- “二进制小数点” 右边的位表示2的分数幂
- 表示有理数：

$$\sum_{k=-j}^i b_k \times 2^k$$

二进制小数：示例

■ 值 表示

$5 \frac{3}{4}$	$= 23/4$	101.11_2	$= 4 + 1 + 1/2 + 1/4$
$2 \frac{7}{8}$	$= 23/8$	10.111_2	$= 2 + 1/2 + 1/4 + 1/8$
$1 \frac{7}{16}$	$= 23/16$	1.0111_2	$= 1 + 1/4 + 1/8 + 1/16$

■ 观测

- 通过右移来表示除以 2（无符号）
- 左移来表示乘以 2
- 形如 $0.111111..._2$ 的数是最接近 1.0 又比 1.0 小的数
 - $1/2 + 1/4 + 1/8 + ... + 1/2^i + ... \rightarrow 1.0$
 - 使用符号 $1.0 - \epsilon$

可表示的数

■ 局限 #1

- 只能准确地表示形如 $x/2^k$ 的数
 - 其他的有理数有循环的位表示
- 值 表示
 - $1/3$ $0.0101010101[01]..._2$
 - $1/5$ $0.001100110011[0011]..._2$
 - $1/10$ $0.0001100110011[0011]..._2$

■ 局限 #2

- 对于 w 位数，只有一个小数点位置的设定
 - 表数范围有限 (对于特别小或特别大的值该怎么办呢？)

本节内容：浮点数

- 背景：二进制小数
- IEEE 浮点标准：定义
- 示例和性质
- 舍入、加法、乘法
- C语言中的浮点数
- 小结

IEEE 浮点表示

■ IEEE 754 标准

- 1985年提出，作为浮点数运算对统一标准
 - 在那之前，有多种各异的格式
- 现在已为所有主流CPU 支持

■ 设计的出发点是数值计算

- 该标准在舍入、上溢和下溢方面设计都很好
- 在硬件设计上很难实现得很快
 - 在定义标准时，数值分析师的意见压倒了硬件设计者的意见

浮点数表示

■ 数字形式:

Example:

$$15213_{10} = (-1)^0 \times 1.1101101101101_2 \times 2^{13}$$

$$(-1)^s M 2^E$$

- 符号位 **s** 决定数字是负数还是正数
- 尾数**M**通常是一个范围为 $[1.0, 2.0)$ 内的小数
- 阶码**E**是2的幂，表明权重

■ 编码

- 最高位 s 就是是符号位 **s**
- exp 字段是 **E** 的编码（但不等于 E）
- frac 字段是 **M** 的编码（但不等于 M）



精度的选项

- 单精度：32 bits
≈ 7 位十进制数字, $10^{\pm 38}$

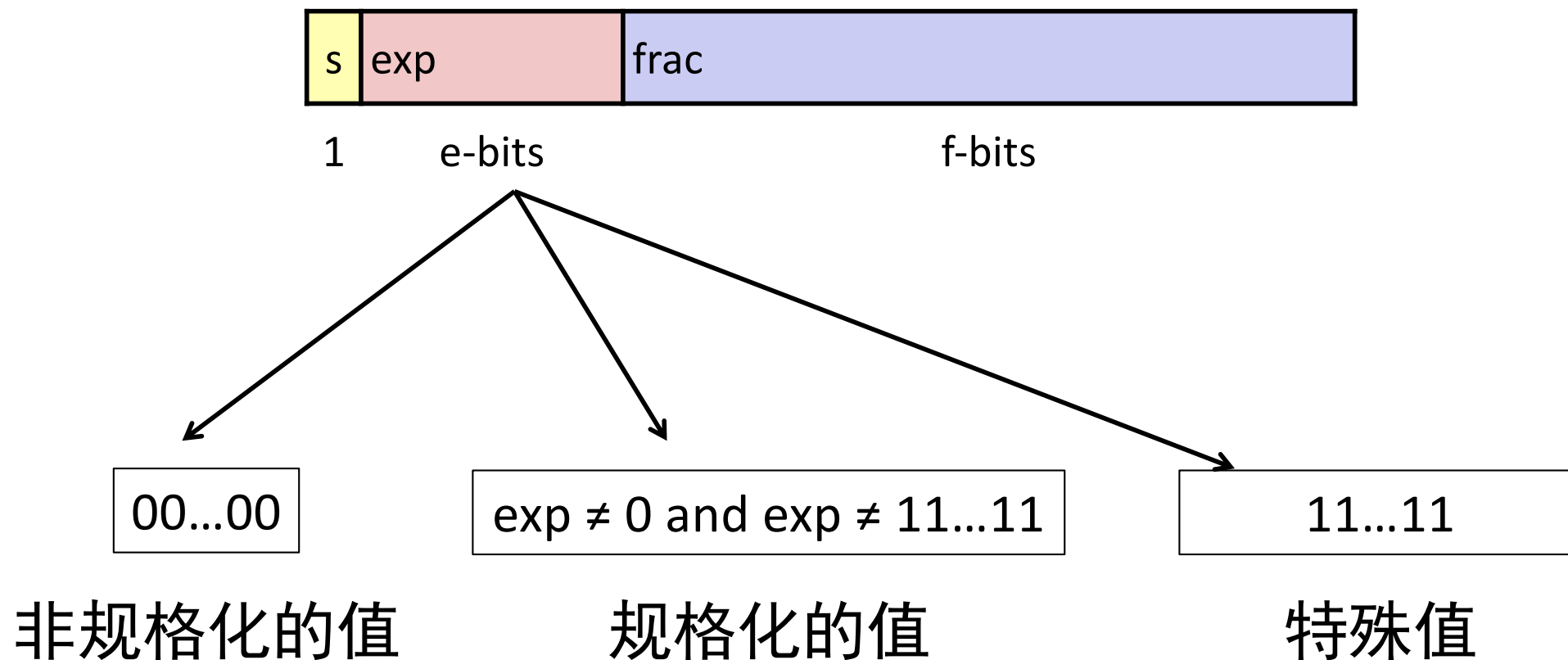


- 双精度：64 bits
≈ 16 位十进制数字, $10^{\pm 308}$



- 其他格式：半精度、四精度

浮点数的三种“种类”



规格化的值

$$v = (-1)^s M 2^E$$

- 当 : **exp** $\neq$ 000...0 且 **exp** $\neq$ 111...1时
- 指数编码为一个偏置的值 : $E = \mathbf{exp} - \text{Bias}$
 - **exp** : 对应无符号值
 - 偏置量 (**bias**) = $2^{k-1} - 1$, 其中 k 是指数的位数数值
 - 单精度 : 127 (**exp**: 1...254, E : -126...127)
 - 双精度 : 1023 (**exp**: 1...2046, E : -1022...1023)
- 尾数编码, 有一个隐含的最高位 : $M = 1.\mathbf{xxx}\dots\mathbf{x}_2$
 - **xxx...x**: **frac**字段对应的位
 - 当 **frac**=000...0 ($M = 1.0$) 时取最小值
 - 当 **frac**=111...1 ($M = 2.0 - \epsilon$) 时取最大值
 - 最高位是“免费”获得的

规格化数编码示例

$$v = (-1)^s M 2^E$$
$$E = \text{exp} - \text{Bias}$$

■ 值: float $F = 15213.0;$

$$15213_{10} = 11101101101101_2$$
$$= 1.1101101101101_2 \times 2^{13}$$

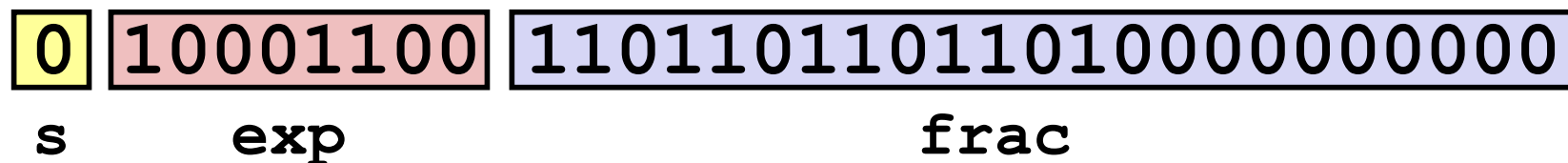
■ 尾数

$$M = 1.\underline{1101101101101}_2$$
$$\text{frac} = \underline{1101101101101}0000000000_2$$

■ 指数

$$E = 13$$
$$\text{Bias} = 127$$
$$\text{exp} = 140 = 10001100_2$$

■ 结果:



非规格化的数

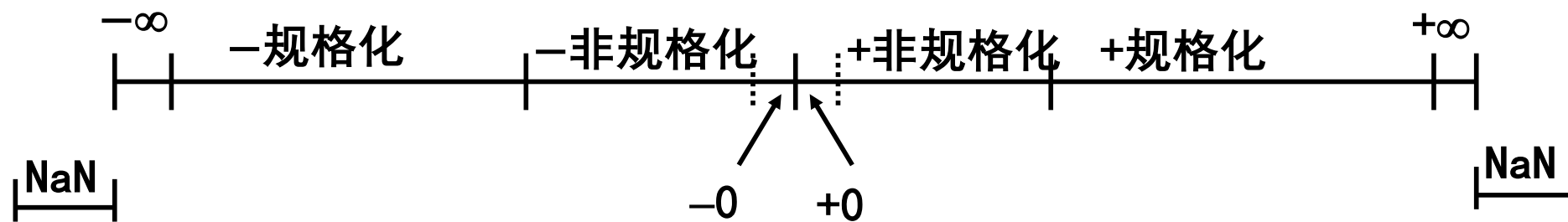
$$v = (-1)^s M 2^E$$
$$E = 1 - \text{Bias}$$

- 条件：exp = 000...0
- 阶码：E = 1 - Bias (而不是 E = 0 - Bias)
- 尾数编码，有一个隐含的最高位0：M = 0.xxx...x2
 - xxx...x: 小数frac字段的位
- 示例
 - exp = 000...0, frac = 000...0
 - 表示值 0
 - 注意0的两种不同的值: +0 和 -0
 - exp = 000...0, frac ≠ 000...0
 - 接近于 0.0 的数字
 - 均匀分布

特殊值

- 条件: $\text{exp} = 111\dots 1$
- 案例: $\text{exp} = 111\dots 1, \text{frac} = 000\dots 0$
 - 表示值 ∞ (无穷大)
 - 运算结果的溢出
 - 有正有负
 - 例如, $1.0/0.0 = -1.0/-0.0 = +\infty, \quad 1.0/-0.0 = -\infty$
- 案例: $\text{exp} = 111\dots 1, \text{frac} \neq 000\dots 0$
 - **Not-a-Number (NaN)**
 - 表示无法确定数值的情况
 - 例如, $\text{sqrt}(-1), \infty - \infty, \infty \times 0$

浮点数编码的可视化



本节内容：浮点数

- 背景：二进制小数
- IEEE 浮点标准：定义
- 示例和性质
- 舍入、加法、乘法
- C语言中的浮点数
- 小结

C语言中的浮点数示例

float: 0xC0A00000

binary: _____



1

8-bits

23-bits

E =

S =

M =

$$v = (-1)^S M 2^E =$$

$$v = (-1)^S M 2^E$$
$$E = \mathbf{exp} - \text{Bias}$$

$$\text{Bias} = 2^{k-1} - 1 = 127$$

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

C语言中的浮点数示例#1

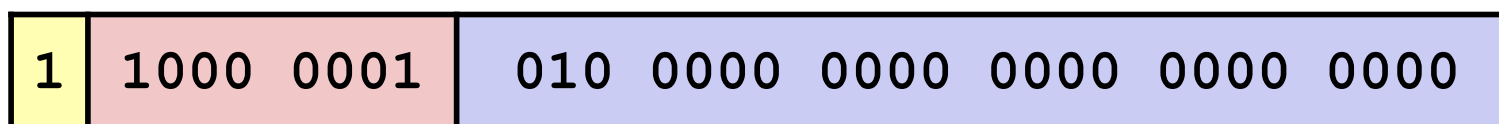
float: 0xC0A00000

$$v = (-1)^s M 2^E$$

$$E = \text{exp} - \text{Bias}$$

$$\text{Bias} = 2^{k-1} - 1 = 127$$

binary 1100 0000 1010 0000 0000 0000 0000 0000



1

8-bits

23-bits

$$E = \text{exp} - \text{Bias} = 129 - 127 = 2 \text{ (decimal)}$$

$$S = 1 \rightarrow \text{负数}$$

$$M = 1.010 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

$$= 1 + 1/4 = 1.25$$

$$v = (-1)^s M 2^E = (-1)^1 * 1.25 * 2^2 = -5$$

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

C语言中的浮点数示例#2

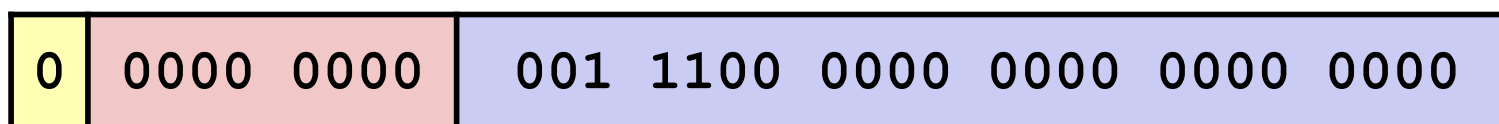
float: 0x001C0000

$$v = (-1)^s M 2^E$$

$$E = 1 - \text{Bias}$$

$$\text{Bias} = 2^{k-1} - 1 = 127$$

binary 0000 0000 0001 1100 0000 0000 0000 0000



1

8-bits

23-bits

$$E = 1 - \text{Bias} = 1 - 127 = -126 \text{ (decimal)}$$

$$S = 0 \rightarrow \text{正数}$$

$$M = 0.001 \ 1100 \ 0000 \ 0000 \ 0000 \ 0000$$

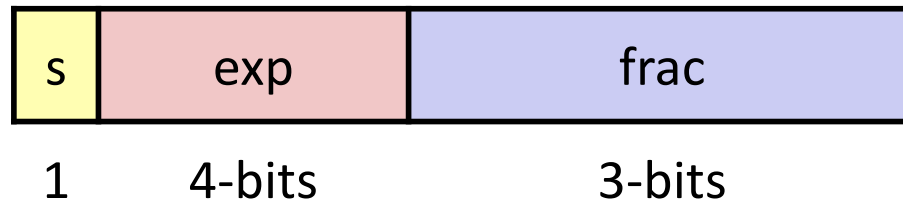
$$= 1/8 + 1/16 + 1/32 = 7/32 = 7 \cdot 2^{-5}$$

$$v = (-1)^s M 2^E = (-1)^0 * 7 \cdot 2^{-5} * 2^{-126} = 7 \cdot 2^{-131}$$

$$\approx 2.571393892 \times 10^{-39}$$

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

一个“微型”浮点数示例



■ 8-bit 浮点数表示

- 最高位为符号位在
- 接下来4位是阶码字段exp，偏置量为 7
- 最后3位为小数字段frac

■ 与 IEEE 格式一致

- 规格化值，非规格化值
- 一些极端数值的表示：0, NaN, 无穷大

动态范围（仅正值）

$$v = (-1)^s M 2^E$$

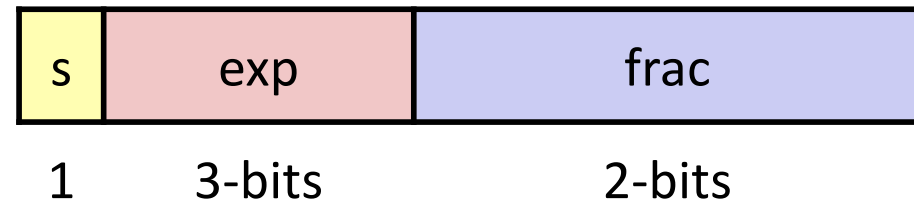
n: $E = \text{Exp} - \text{Bias}$
d: $E = 1 - \text{Bias}$

	s	exp	frac	E	Value	
非规格化值	0	0000	000	-6	0	
	0	0000	001	-6	$1/8 * 1/64 = 1/512$	closest to zero
	0	0000	010	-6	$2/8 * 1/64 = 2/512$	
	...					
	0	0000	110	-6	$6/8 * 1/64 = 6/512$	
	0	0000	111	-6	$7/8 * 1/64 = 7/512$	largest denorm
规格化值	0	0001	000	-6	$8/8 * 1/64 = 8/512$	smallest norm
	0	0001	001	-6	$9/8 * 1/64 = 9/512$	
	...					
	0	0110	110	-1	$14/8 * 1/2 = 14/16$	
	0	0110	111	-1	$15/8 * 1/2 = 15/16$	closest to 1 below
	0	0111	000	0	$8/8 * 1 = 1$	
	0	0111	001	0	$9/8 * 1 = 9/8$	closest to 1 above
	0	0111	010	0	$10/8 * 1 = 10/8$	
	...					
	0	1110	110	7	$14/8 * 128 = 224$	
	0	1110	111	7	$15/8 * 128 = 240$	largest norm
	0	1111	000	n/a	inf	

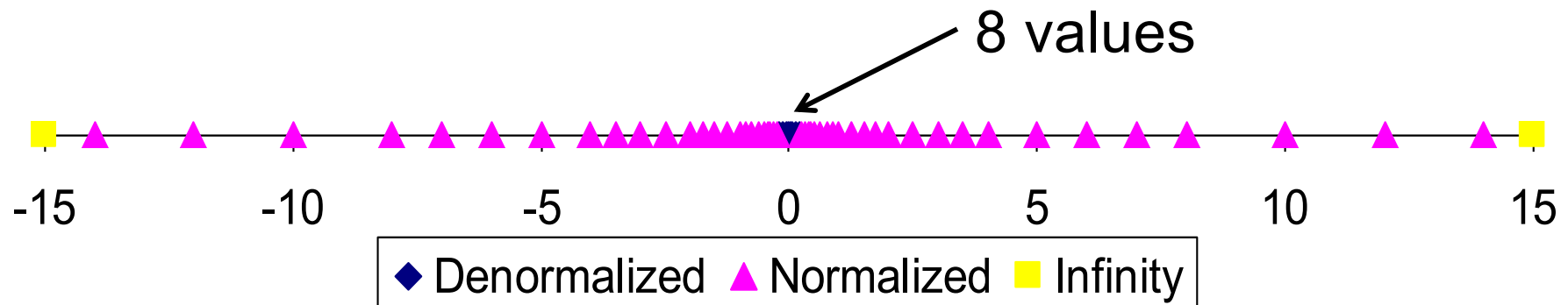
数值的分布

■ 6-bit 类IEEE 格式

- $e = 3$ exp 位
- $f = 2$ frac 位
- 偏置量为 $2^{3-1}-1 = 3$



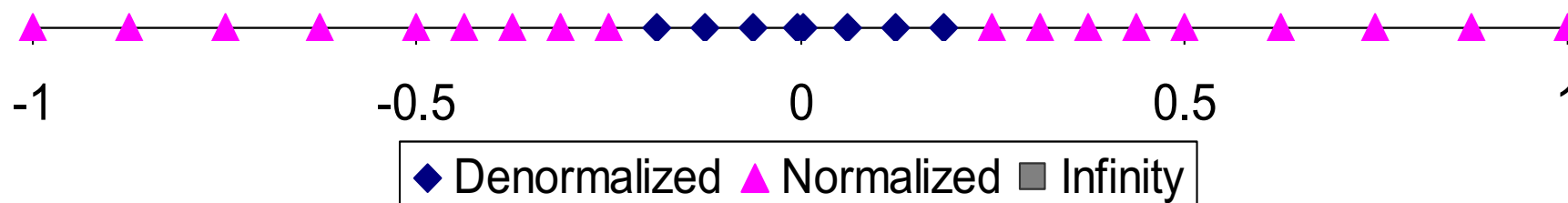
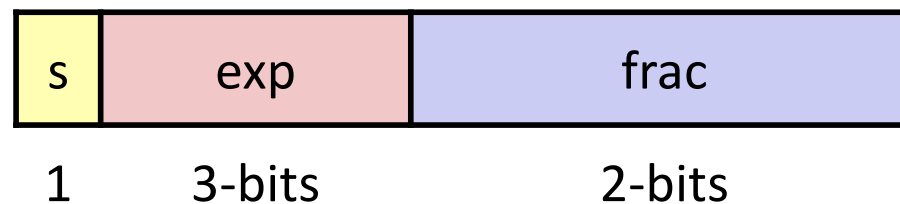
■ 注意分布如何向零变得更密集



值的分布（特写视图）

■ 6-bit 类IEEE 格式

- $e = 3$ exp 位
- $f = 2$ frac 位
- 偏置量3



有趣的数

{单精度, 双精度}

描述	阶码	小数	数值
■ 零	00...00	00...00	0.0
■ 最小的正非规范化值	00...00	00...01	$2^{-\{23,52\}} \times 2^{-\{126,1022\}}$
■ 单精度 $\approx 1.4 \times 10^{-45}$ ■ 双精度 $\approx 4.9 \times 10^{-324}$			
■ 最大的非规格化值	00...00	11...11	$(1.0 - \epsilon) \times 2^{-\{126,1022\}}$
■ 单精度 $\approx 1.18 \times 10^{-38}$ ■ 双精度 $\approx 2.2 \times 10^{-308}$			
■ 最小的正规格化值	00...01	00...00	$1.0 \times 2^{-\{126,1022\}}$
■ Just larger than largest denormalized			
■ 一	01...11	00...00	1.0
■ 最大的规格化值	11...10	11...11	$(2.0 - \epsilon) \times 2^{\{127,1023\}}$
■ 单精度 $\approx 3.4 \times 10^{38}$ ■ 双精度 $\approx 1.8 \times 10^{308}$			

IEEE 编码的特殊属性

- 浮点数0与整型0的表示相同
 - 所有位 = 0
- （几乎）可以使用无符号整数比较
 - 先比较符号位
 - 必须考虑 $-0 = +0$
 - NaN 有点麻烦
 - 大于任何其他数值
 - 该如何产生比较结果呢？
 - 其他都没有问题
 - 非规格化数 vs 规格化数
 - 规格化 vs 无穷大

本节内容：浮点数

- 背景：二进制小数
- IEEE 浮点标准：定义
- 示例和性质
- 舍入、加法、乘法
- C语言中的浮点数
- 小结

浮点数运算：基本思想

■ $x +_f y = \text{Round}(x + y)$

■ $x \times_f y = \text{Round}(x \times y)$

■ 基本思想

- 首先计算精确结果
- 使之适配到期望的精度
 - 如果指数太大，可能会溢出
 - 可能需要舍入来适配小数frac字段

舍入

■ 舍入模式（以 \$ 美元舍入为例）

	\$1.40	\$1.60	\$1.50	\$2.50	-\$1.50
■ 向0舍入	\$1 ↓	\$1 ↓	\$1 ↓	\$2 ↓	-\$1 ↑
■ 向下舍入($-\infty$)	\$1 ↓	\$1 ↓	\$1 ↓	\$2 ↓	-\$2 ↓
■ 向上舍入($+\infty$)	\$2 ↑	\$2 ↑	\$2 ↑	\$3 ↑	-\$1 ↑
■ *向偶数舍入(默认)	\$1 ↓	\$2 ↑	\$2 ↑	\$2 ↓	-\$2 ↓

*将数字向上或者向下舍入，使得结果的最低有效位是偶数

详细观察向偶数舍入

■ 缺省的舍入模式

- 所有其他的模式都是统计学上有偏差的
 - 一组正数的和将始终被高估或低估

■ 十进制数的向偶数舍入

- 当需要舍入的值刚好在两个合法值中间时
 - 向最低位为偶数的方向舍入
- 例如，舍入到百分位

7.8949999	7.89	(小于中间值)
7.8950001	7.90	(大于中间值)
7.8950000	7.90	(中间值—向上舍入)
7.8850000	7.88	(中间值—向下舍入)

二进制数的舍入

■ 二进制小数

- “偶数”是指最低位为0
- “刚好在中间”是指待舍入位置点的右边的位= $100..._2$

■ 示例

- 舍入到最接近的 $1/4$ （二进制小数点右侧 2 位）

值	二进制数	舍入	处理	舍入后的值
$2 \frac{3}{32}$	$10.00\textcolor{red}{011}_2$	10.00_2	($<1/2$ —向下)	2
$2 \frac{3}{16}$	$10.00\textcolor{red}{110}_2$	10.01_2	($>1/2$ —向上)	$2 \frac{1}{4}$
$2 \frac{7}{8}$	$10.11\textcolor{red}{100}_2$	11.00_2	($1/2$ —向上)	3
$2 \frac{5}{8}$	$10.10\textcolor{red}{100}_2$	10.10_2	($1/2$ —向下)	$2 \frac{1}{2}$

浮点加法

■ $(-1)^{s1} M1 2^{E1} + (-1)^{s2} M2 2^{E2}$

■ 假设 $E1 > E2$

■ 精确的结果： $(-1)^s M 2^E$

■ 符号 s , 尾数 M :

▪ 有符号数对齐&相加的结果

■ 阶码 E : $E1$

修正

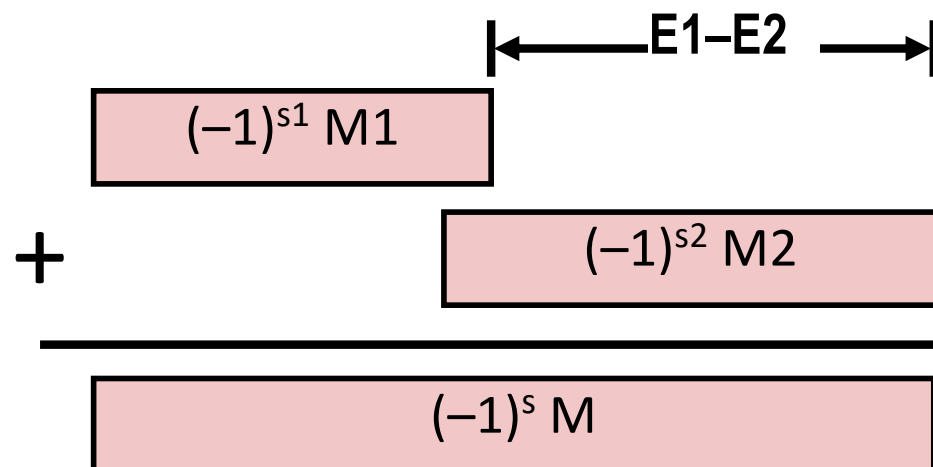
■ 如果 $M \geq 2$, M 右移, E 增加

■ 如果 $M < 1$, M 左移 k 位, E 减 k

■ 如果 E 超出范围, 则溢出

■ 将 M 舍入以适合 **frac** 字段的精度

二进制小数点要对齐



$$\begin{aligned} 1.010 \cdot 2^2 + 1.110 \cdot 2^3 &= (0.1010 + 1.1100) \cdot 2^3 \\ &= 10.0110 \cdot 2^3 = 1.00110 \cdot 2^4 = 1.010 \cdot 2^4 \end{aligned}$$

浮点加法的数学性质

■ 交换律?

Yes

■ 结合律?

No

- 可能溢出，不准确的舍入

- $(3.14 + 1e20) - 1e20 = 0, \quad 3.14 + (1e20 - 1e20) = 3.14$

■ 单调性

- $a \geq b \Rightarrow a + c \geq b + c?$

Yes

- 除了无穷大和 NaN 以外，都成立

$$+\infty - \infty = \text{NaN}$$

$$\text{NaN} + x = \text{NaN}$$

浮点乘法

■ $(-1)^{s1} M1 2^{E1} \times (-1)^{s2} M2 2^{E2}$

■ 精确值： $(-1)^s M 2^E$

■ 符号 s : $s1 \wedge s2$

■ 尾数 M : $M1 \times M2$

■ 阶码 E : $E1 + E2$

■ 修正

- 如果 $M \geq 2$, M 右移, E 增加
- 如果 E 超出范围, 则溢出
- 将 M 舍入以适合**frac**的精度

■ 实现

- 最大的工作量是将尾数相乘

4 位尾数: $1.010 \times 2^2 \times 1.110 \times 2^3 = 1\textcolor{red}{0}.0011 \times 2^5$
 $= 1.000\textcolor{red}{11} \times 2^6 = 1.00\textcolor{red}{1} \times 2^6$

浮点乘法的数学性质

■ 乘法交换律?

Yes

■ 乘法结合律?

No

- 有溢出的可能, 舍入有不准确性

- 例: $(1e20 * 1e20) * 1e-20 = \text{inf}$, $1e20 * (1e20 * 1e-20) = 1e20$

■ 1 是乘法标识吗?

■ 乘法在加法上可分配吗?

No

- 有溢出的可能性, 舍入有不准确性

- $1e20 * (1e20 - 1e20) = 0.0$, $1e20 * 1e20 - 1e20 * 1e20 = \text{NaN}$

■ 单调性

- $a \geq b \ \& \ c \geq 0 \Rightarrow a * c \geq b * c$?

- 除了无穷大和 NaN 以外, 都成立

今日：浮点数

- 背景：二进制小数
- IEEE 浮点标准：定义
- 示例和属性
- 舍入、加法、乘法
- C语言中的浮点数
- 小结

C语言中的浮点数

■ C 语言中有两种浮点数

- `float` 单精度
- `double` 双精度

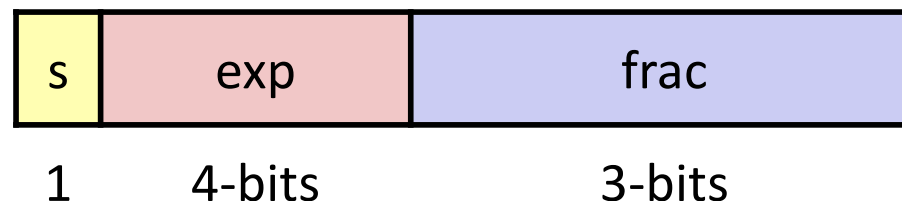
■ 转换/强制类型转换

- `int`, `float`, 和 `double` 之间的转换会改变位表示
- `double/float → int`
 - 截断小数部分
 - 类似于向零舍入
 - 超出表示范围或 NaN 时，行为未定义：通常设置为 TMin
- `int → double`
 - 精确转换，只要 $\text{int} \leq 53$ 位
- `int → float`
 - 可能会进行舍入

创建浮点数

■ 步骤

- 前导为1的规格化值
- 舍入以适合小数
- 后规格化来处理舍入的影响



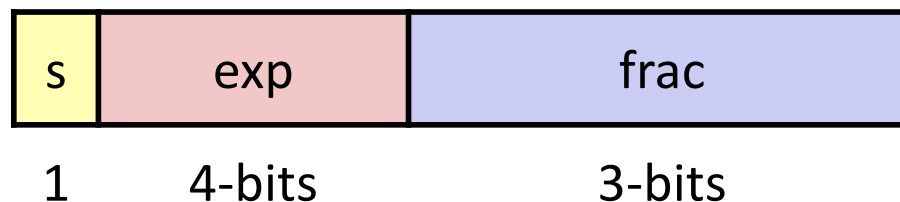
■ 案例分析

- 将 8 位无符号数转换为微小的浮点格式

示例

128	10000000
15	00001101
33	00010001
35	00010011
138	10001010
63	00111111

规格化值

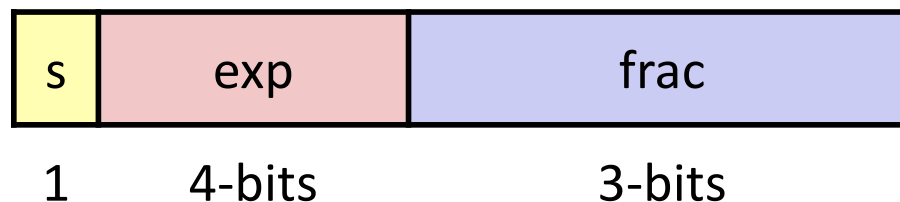


■ 需求

- 设置二进制小数点，使形式为 1.xxxxx 的数字
- 调整所有的前导1
 - 递减阶码为左移

值	二进制	小数M	阶码E
128	10000000	1.0000000	7
15	00001101	1.1010000	3
17	00010001	1.0001000	4
19	00010011	1.0011000	4
138	10001010	1.0001010	7
63	00111111	1.1111100	5

规格化之后



■ 问题

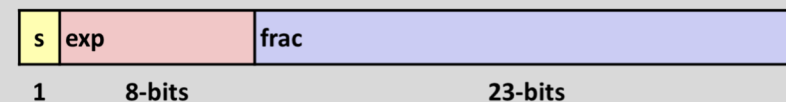
- 舍入可能导致溢出
- 通过右移一次并增加阶码来处理

值	小数M	舍入	阶码E	调整	结果
128	1.0000000	1.000	7		128
15	1.1010000	1.101	3		15
17	1.0001000	1.000	4		16
19	1.0011000	1.010	4		20
138	1.0001010	1.001	7		134
63	1.1111100	10.000	5	1.000/6	64

浮点数小结

- IEEE 浮点数有明确的数学性质
- 表示形如 $M \times 2^E$ 的数字
- 可以依据规则推测运算的结果，与实现无关
 - 好像是按照完美精度进行计算，再进行舍入
- 与真实的算术计算是不同的
 - 没有结合律和分配律
 - 对编译器和严肃的数值应用程序员来说，增加了很多困难

Single precision: 32 bits



Double precision: 64 bits

